Parallel Decoder Transformer: Planner-Seeded Latent Coordination for Synchronized Parallel Generation

Logan Robbins
Independent Researcher
ljrweb@gmail.com

March 9, 2026

Abstract

Autoregressive language models can often identify parallel subproblems, but standard decoding exposes only a single left-to-right output interface. External orchestration methods can launch multiple prompts concurrently, yet they provide no model-internal state through which those generations can synchronize, resolve ownership, or wait for missing information. We present the **Parallel Decoder Transformer (PDT)**, a frozen-trunk architecture that augments a decoder with a planner-seeded latent workspace and a synchronized multi-stream output protocol. Before any stream emits tokens, a mandatory prompt-time planner predicts fixed latent plan slots and projects them as snapshot 0 on an embeddings-only Dynamic Notes Bus. During decoding, each stream reads the visible notes window through Speculative Note Conditioning (SNC), emits provisional token blocks and latent summaries, and advances only when agreement logic determines that the current shared state is sufficient for continued parallel generation. Coverage heads track plan-item ownership, while rollback handles incoherent or premature commits. PDT therefore shifts parallel task decomposition from an external prompting strategy to a model-internal coordination mechanism over the output interface of a frozen language model.

1 Introduction

Large language models frequently encounter prompts whose natural solution is not a single uninterrupted chain, but a set of partially independent sections, subquestions, or arguments. A model may internally recognize that decomposition, yet standard autoregressive decoding exposes only one causal output stream. The consequence is structural rather than merely computational: even when multiple sections could in principle be developed concurrently, the model must serialize them through a single textual channel.

External decomposition methods partially relieve that constraint by prompting for an outline and then spawning multiple generations in parallel [11, 9]. Those methods can improve throughput and encourage modularity, but they do not create model-internal shared state. Once the work is split across separate calls, no stream can directly know whether a sibling stream has already established a key fact, claimed ownership of a section, or left a dependency unresolved. We refer to the resulting failure mode as **coherence drift**: parallel branches remain semantically related, but their local continuations can become redundant, contradictory, or prematurely specific.

This paper introduces the **Parallel Decoder Transformer (PDT)**, a decoder-only architecture that moves both decomposition and synchronization inside the model. PDT begins with a mandatory prompt-time latent planner that seeds a shared snapshot-0 workspace before any stream emits tokens. Parallel streams then decode against that workspace through **Speculative Note Conditioning (SNC)**, write provisional latent summaries at synchronized block boundaries, and use coverage plus agreement logic to decide whether the current multi-stream state may be committed and extended.

The key claim of PDT is not that parallel streams are always faster, but that a decoder can be given a mechanism for **synchronized parallel generation**: streams may emit concurrently, but they only commit and continue when shared latent state is sufficient for cross-stream consistency.

Under that interpretation, PDT is not just a decoding heuristic. It is a model-internal coordination protocol over the output interface of a frozen language model. The contribution is therefore not merely parallelism, but a way for a decoder to decompose a task into parallel streams, exchange latent state, and advance the parallel frontier only when the shared workspace supports safe continuation.

1.1 Contributions

1. **Planner-seeded multi-stream generation protocol.** A mandatory prompt-time planner maps the prompt into fixed latent plan slots and initializes a shared snapshot-0 workspace before any stream emits tokens.
2. **Embeddings-only coordination bus for synchronized continuation.** Parallel streams read a lagged latent workspace during token emission, emit provisional summaries at block boundaries, and continue only when agreement deems the shared state sufficient for the next block.
3. **Ownership-aware commit control.** Coverage, ownership, and agreement jointly determine whether provisional content should be committed, withheld, or regenerated, enabling coordinated parallel generation without raw-text inter-stream exchange.
4. **Frozen-trunk realization.** The full coordination stack attaches to a frozen decoder through lightweight sidecar modules, preserving the base model while adding planner, bus, and synchronization behavior.

2 Related Work

2.1 Token-Level Acceleration Versus Coordination

Standard autoregressive decoding generates tokens sequentially as $p(x_t \mid x_{<t})$. Token-level acceleration methods such as Speculative Decoding [7], Blockwise Parallel Decoding [14], Medusa [1], EAGLE [15], Lookahead Decoding [8], and related analyses of speculative trade-offs [19, 20] improve local efficiency by predicting or verifying multiple future tokens. More recent work also studies concurrent attention and interleaved planning mechanisms for faster generation [12, 18].

These methods are important reference points, but they solve a different problem. They accelerate a single causal stream or speculate over future tokens within one stream. PDT is not primarily a faster decoding scheme; it is a model-internal synchronization mechanism for deciding when multiple partial generations may safely continue in parallel.

2.2 External Prompt-Level Decomposition

Prompt-level approaches such as Skeleton-of-Thought (SoT) [11] and PSLM [9] ask a model for a decomposition and then launch multiple prompted generations. Their central advantage is semantic modularity: different calls can elaborate different parts of a solution. Their central limitation is where coordination lives. Shared state remains outside the model and is mediated by prompt text, API orchestration, or post-hoc merging. As a result, these systems can decompose work but cannot provide an internal continuation rule for when sibling streams have supplied enough information for safe further generation.

PDT is closest in motivation to this family of methods, but differs in mechanism. The decomposition prior, the shared workspace, and the continuation gate all live inside one decoder rather than in an external controller.

2.3 Shared Workspaces, Blackboard Systems, and Latent Communication

A growing literature coordinates multiple model calls or agents through shared workspaces, graph structures, or latent communication channels. GPTSwarm [21], Mixture-of-Agents [17], blackboard-style systems [3, 13], and topology-adaptive agent systems such as AMAS [6] all point toward coordination as a systems problem rather than a single-prompt problem. Earlier work on shared neural workspaces [2] provides a conceptual precedent for a global coordination substrate, while LatentMAS [22] demonstrates the value of latent rather than text-mediated communication.

PDT shares that intuition but places the workspace inside a single frozen decoder and binds it directly to the output interface. Its streams are not separate prompted agents. They are coordinated generation branches of one model, each with its own local cache but a common latent synchronization workspace.

2.4 Parameter-Efficient Adaptation and Distillation

Because modifying all parameters of a large decoder is computationally expensive, PDT follows the parameter-efficient adaptation tradition of LoRA [5] and adapters [4]. Distillation and learning-with-privileged-information frameworks [10, 16] further motivate training auxiliary coordination modules with teacher signals that may be unavailable or undesirable at inference time.

These techniques matter here not only for efficiency, but because they make PDT a realistic architectural extension of a frozen model. The planner, synchronization workspace, and commit-control heads can be added without rewriting the underlying language model.

3 Architecture

The Parallel Decoder Transformer turns one frozen decoder into K coordinated output streams. Its central design decision is that decomposition and cross-stream state are **model-internal**. The prompt is planned once, the resulting latent plan initializes a shared workspace, and each stream decodes against that workspace rather than against a separate prompt string.

3.1 Frozen Trunk Topology

We initialize PDT from a pre-trained decoder-only backbone parameterized by θ_{pre} and freeze all weights in θ_{pre}. Trainable parameters ϕ are introduced as a lightweight coordination stack. The full parameter set is

$$\Theta = \theta_{\text{pre}} \cup \phi. \tag{1}$$

The trainable subset ϕ contains four components:

- **Stream adapters**, inserted into selected transformer blocks to provide stream-specific conditioning.
- **SNC backends**, implemented as cross-attention layers that read from the shared latent workspace.
- **Planner and notes modules**, including a fixed-slot planner head, a shared plan-embedding matrix, and a projection into the notes space used by the bus.
- **Auxiliary control heads**, including note-emission, coverage, agreement, and stream-classification heads.

All streams share the same frozen trunk parameters, but each stream maintains its own KV cache, adapter state, and decode position. The base language model therefore remains canonical, while coordination is expressed through sidecar modules rather than through changes to pre-trained weights.

3.2 Prompt-Time Latent Planner

Let $\mathbf{H}_x \in \mathbb{R}^{T \times d}$ denote the hidden states of the prompt under the frozen trunk. PDT performs a mandatory planning pass before any output tokens are generated. The planner predicts logits over S latent plan slots and a shared plan vocabulary of size V_p:

$$\blacksquare = \text{PlannerHead}(\mathbf{H}_x) \in \mathbb{R}^{S \times V_p}, \qquad z_i = \arg\max_a \blacksquare_{i,a}. \tag{2}$$

In the current implementation, $S = 16$ and $V_p = 65{,}536$.

The latent slot identifiers $z_{1:S}$ are not language-model tokens. They index a shared latent plan vocabulary that is also used by the plan embedding matrix, the canonical plan catalog, and coverage targets. Active slots are re-embedded and projected into notes space:

$$\mathbf{e}_i = E_{\text{plan}}[z_i], \qquad \mathbf{n}_0^{\text{plan}} = \text{Norm}\left(\frac{1}{|M|} \sum_{i \in M} W_{\text{plan}} \mathbf{e}_i \right), \tag{3}$$

where M denotes the active planner slots.

The resulting vector $\mathbf{n}_0^{\text{plan}}$ is broadcast as **snapshot 0** on the Dynamic Notes Bus before any stream emits tokens. The planner does not merely assign semantic subtopics. It initializes the shared coordination state against which subsequent continuation decisions are made. Snapshot 0 therefore serves both as a decomposition prior and as the first synchronization contract among streams.

In PDT, parallel generation does not begin from independent empty states. It begins from a common latent commitment structure.

3.3 Dynamic Notes Bus as Synchronization Workspace

The **Dynamic Notes Bus** is PDT's shared latent workspace. It is an embeddings-only, versioned store of planner and stream summaries. Textual plans and rendered notes may be used for dataset construction, observability, or supervision, but inference-time coordination uses only note embeddings.

We describe the runtime at two timescales:

- τ: the number of tokens in each provisional block emitted per stream between synchronization decisions,
- Δ: the reveal delay between a note being emitted and becoming visible to sibling streams,
- H: the rollback or commit horizon over which recently generated provisional content may be withheld or regenerated.

Let $\widehat{\mathbf{n}}_u^{(j)}$ denote the provisional note emitted by stream j at synchronization round u, and let B be the workspace budget. The visible notes window for stream k at round v is

$$\mathscr{W}_v^{(k)} = \text{Window}\left(\left\{\mathbf{n}_0^{\text{plan}}\right\} \cup \left\{\widehat{\mathbf{n}}_u^{(j)} : j \in [K],\ u \leq v - \Delta\right\}, B\right). \tag{4}$$

The Dynamic Notes Bus is therefore not just an auxiliary memory. It is the synchronization workspace that determines whether provisional multi-stream generation may be committed and extended.

3.4 Synchronized Block Emission Protocol

PDT runs in synchronized rounds rather than as unconstrained free-running streams. At synchronization round v:

1. The planner-seeded bus exposes the visible notes window $\mathscr{W}_v^{(k)}$ to each active stream k.
2. Each stream emits a provisional block of τ tokens conditioned on its local cache and the visible workspace:

$$\widehat{\mathbf{y}}_v^{(k)} = \left(\hat{y}_{v,1}^{(k)}, \ldots, \hat{y}_{v,\tau}^{(k)}\right). \tag{5}$$

3. At the end of the block, stream k emits a provisional latent note $\widehat{\mathbf{n}}_v^{(k)}$ summarizing what it has established, which plan items it believes it owns, and which dependencies remain unresolved.
4. Coverage and agreement heads evaluate whether the provisional block is consistent with the shared plan and whether sibling state is sufficient for further continuation.
5. If agreement passes, the block is committed and the new note snapshot becomes visible after delay Δ.
6. If agreement fails, the system withholds commit, stalls a subset of streams, or rolls back and regenerates within horizon H using fresher shared context.

This protocol is the core mechanism of PDT. Streams are allowed to generate in parallel only up to a provisional block boundary; continuation beyond that boundary is gated by agreement over the shared latent workspace.

3.5 Speculative Note Conditioning Cross-Attention

SNC is the read mechanism over the visible workspace. During token emission inside round v, stream k conditions on $\mathscr{W}_v^{(k)}$ at every decode step. Let $\mathbf{H}_{v,u}^{(k)}$ denote the hidden state for token position $u \in \{1, \ldots, \tau\}$ in round v. The SNC projections are

$$\mathbf{Q}_{v,u}^{(k)} = \mathbf{H}_{v,u}^{(k)} \mathbf{W}_Q, \qquad \mathbf{K}_v^{(k)} = \mathscr{W}_v^{(k)} \mathbf{W}_K, \qquad \mathbf{V}_v^{(k)} = \mathscr{W}_v^{(k)} \mathbf{W}_V, \tag{6}$$

with learned parameters $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \phi$.

The resulting SNC context vector $\mathbf{C}_{v,u}^{(k)}$ is injected through a trust-gated residual:

$$\widetilde{\mathbf{H}}_{v,u}^{(k)} = \mathbf{H}_{v,u}^{(k)} + \lambda_l \cdot \mathbf{C}_{v,u}^{(k)} \mathbf{W}_O, \tag{7}$$

where $\lambda_l \in [0,1]$ is a learned residual gate for the instrumented layer. The gate preserves the magnitude statistics of the frozen trunk early in training and allows coordination to become stronger only when the auxiliary path is reliable.

SNC provides continuous low-bandwidth conditioning during token emission, but synchronization decisions are made only at block boundaries. PDT therefore separates two timescales: **token-time latent conditioning** and **block-time continuation control**.

3.6 Provisional Writes, Ownership, and Commit Readiness

At the end of each provisional block, stream k writes a latent summary back to the bus:

$$\widehat{\mathbf{n}}_v^{(k)} = \text{SpeculationHead}\left(\widetilde{\mathbf{H}}_{v,\tau}^{(k)}\right). \tag{8}$$

That note is intended to summarize the stream's newly established content, its unresolved dependencies, and its current ownership claims.

Coverage is predicted against the planner-seeded plan catalog:

$$\mathbf{c}_v^{(k)} = \text{CoverageHead}\left(\widetilde{\mathbf{H}}_{v,\tau}^{(k)}, E_{\text{plan}}(z_{1:S}), m_{\text{plan}}\right), \tag{9}$$

where m_{plan} masks inactive or padded plan items.

Because coverage targets are stream-qualified canonical plan items, they track more than semantic relevance. They also track ownership and non-overlap. A provisional block is not ready to commit merely because it is fluent; it must also preserve ownership consistency with respect to the planner-seeded plan catalog.

3.7 Agreement-Gated Commit and Rollback Control

Agreement in PDT is not only a rollback trigger after a bad block. It is the permission signal for the next parallel block. For each active stream k at round v, PDT predicts a **readiness score**

$$r_v^{(k)} = \text{AgreeHead}\left(\widetilde{\mathbf{H}}_{v,\tau}^{(k)}, \mathscr{W}_v^{(k)}, \mathbf{c}_v^{(k)}, \widehat{\mathbf{n}}_v^{(k)}\right), \tag{10}$$

which estimates whether stream k has enough synchronized information about sibling streams to safely commit its current block and continue.

The base continuation rule is a global gate over active streams:

$$A_v = \mathbf{1}\left[\min_{k \in \mathscr{A}_v} r_v^{(k)} > \gamma\right], \tag{11}$$

where $\mathscr{A}_v$ is the set of active streams at round v and γ is a learned or tuned threshold.

If $A_v = 1$, PDT commits the provisional blocks $\widehat{\mathbf{y}}_v^{(k)}$ and schedules the new notes $\widehat{\mathbf{n}}_v^{(k)}$ to become visible after delay Δ. If $A_v = 0$, PDT may roll back only the streams whose readiness falls below threshold and let stable streams keep their current commit point. In that selective policy, the global gate acts as a synchronization checkpoint while rollback is applied per stream.

In PDT, agreement is the gate that decides whether the system may advance the parallel frontier.

A richer alternative would replace the scalar gate with pairwise or graph-structured compatibility scores between streams. We view that extension as important future work, but the scalar readiness formulation plus selective rollback is sufficient to define the current synchronized continuation protocol.

3.8 Inference-Time Coordination Loop

The full inference procedure is:

1. Encode the prompt with the frozen trunk.
2. Run the planner to predict latent plan slots $z_{1:S}$.
3. Project the latent plan into notes space and publish snapshot 0 on the Dynamic Notes Bus.
4. For synchronization rounds $v = 1, 2, \ldots$ until all streams terminate:
   (a) Expose the visible notes window $\mathscr{W}_v^{(k)}$ to each active stream.
   (b) Let each active stream emit τ provisional tokens using its local cache plus SNC conditioning.
   (c) Let each active stream emit a provisional latent note $\widehat{\mathbf{n}}_v^{(k)}$.
   (d) Compute coverage $\mathbf{c}_v^{(k)}$ and readiness $r_v^{(k)}$ for each active stream.
   (e) If $A_v = 1$, commit the current block and schedule the new notes for publication after delay Δ.
   (f) Otherwise, stall, selectively withhold, or roll back within horizon H, then regenerate with updated visible context.
5. Merge committed outputs according to planner ownership, section order, and stream-completion state.

The inference loop makes the architectural claim explicit: PDT is a decode → summarize → agree → commit → continue protocol.

3.9 Inference Serving Contract

PDT uses a stride-commit serving contract. Per-stream provisional tokens are buffered inside each stride and are released only after stride-level agreement/rollback resolution. A practical API returns both (i) structured per-stream commit artifacts and (ii) a merged user-facing answer.

```
Prompt
  +-> planner seed (snapshot 0)
        +-> per-stream provisional decode (stride v)
              +-> latent note summaries + readiness
                    +-> commit/rollback decision
                          |-- commit: release committed_text_block(stream, stride)
                          +-- rollback: regenerate failing streams
                                   |
                           final merged answer
```

Coherence in PDT comes from latent communication and synchronized commit decisions, not from direct raw-token sharing across streams.

3.10 Near-Term Use Case: Parallelized Knowledge-Structured Responses

The strongest near-term product setting is prompts with explicit topical substructure (e.g., historical overviews, multi-facet knowledge synthesis, and sectioned explanatory answers). In these cases, planner-seeded ownership priors, latent note exchange, and stride-commit synchronization naturally map onto section-oriented generation and reduce cross-stream contradiction compared to unconstrained live token streaming.

3.11 Parameter-Efficient Curriculum

Training a coordination mechanism on a frozen trunk is unstable if all modules are enabled at once. PDT therefore uses a staged curriculum:

- **Stage 0 (planner pretraining).** Train the planner head, plan embedding pathway, and initial notes projection against latent planner targets and the snapshot-0 plan contract.
- **Stage 1 (stream bootstrap).** Unfreeze stream adapters and SNC cross-attention so streams learn stream-specific conditioning against fixed teacher workspaces.
- **Stage 2 (bus enablement).** Train note-emission modules so streams begin writing learned latent summaries into the versioned bus while the planner seed remains stable.
- **Stage 3 (commit control).** Train coverage and agreement heads so ownership consistency and continuation sufficiency become part of the generation policy.

The accompanying dataset and supervision contract reflect the same runtime assumptions. Snapshot 0 is explicitly materialized from the planner contract, canonical plan items are hashed into the shared latent vocabulary, and language-model supervision is masked so each stream is trained only on its assigned section.

4 Conclusion

PDT reframes parallel generation as a synchronization problem inside the decoder. Its central contribution is not simply to decompose tasks, but to let a model emit multiple provisional streams, exchange latent summaries, and decide when those streams have enough shared state to continue in parallel without collapsing back to text-mediated orchestration.

The architecture's key pieces serve one unified protocol. A mandatory prompt-time planner initializes a shared latent commitment structure; SNC provides continuous token-time access to the visible workspace; coverage tracks ownership and overlap with respect to the planner-seeded catalog; and agreement gates block-level commit, withholding, rollback, and continuation. Together they define a planner-seeded, model-internal coordination mechanism over the output interface of a frozen language model.

PDT therefore shifts the question from "How can multiple prompts be run at once?" to "How can one decoder maintain synchronized multi-stream state while generating?" That shift is the conceptual center of the method and the primary claim of this paper.

4.1 Future Directions

1. **Agreement as continuation sufficiency.** The most immediate evaluation target is whether readiness scores actually predict safe continuation, not merely whether they detect bad commits after the fact.
2. **Dependency-aware synchronization.** Scalar readiness can be generalized to pairwise or graph-structured compatibility so streams gate continuation only on the siblings they depend on.
3. **Adaptive block size τ.** Some streams may be able to advance farther than others when their dependencies are weak; adaptive block sizing would relax the current fixed-round protocol.
4. **Ownership-aware merge policies.** Final composition can itself become a learned function of planner slots, coverage state, and stream completion rather than a fixed section-order merge.
5. **Scaling planner capacity.** Richer ownership semantics, adaptive slot counts, and larger numbers of streams may allow the synchronization workspace to represent more complex task structures.

References

[1] T. Cai et al. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[2] A. Goyal et al. Coordination among neural modules through a shared workspace. *arXiv preprint arXiv:2103.01197*, 2021.

[3] Bochen Han and Songmao Zhang. Exploring advanced LLM multi-agent systems based on blackboard architecture. *arXiv preprint arXiv:2507.01701*, 2025.

[4] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morri, Andrea De Coro, Sergei Vassilvitskii, Ariel Fisher, and Deep Ganguli. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, 2019.

[5] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[6] Hui Yi Leong, Yuheng Li, Yuqing Wu, Wenwen Ouyang, Wei Zhu, and Jiechao Gao. AMAS: Adaptively determining communication topology for LLM-based multi-agent system. *arXiv preprint arXiv:2510.01617*, 2025.

[7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.

[8] Y. Li, F. Wei, C. Zhang, and H. Zhang. Break the sequential dependency of llm inference using lookahead decoding. In *Proceedings of ICML 2024*, 2024. arXiv:2402.02057.

[9] X. Liu et al. Pslm: Parallel generation of text and speech with llms. In *Findings of EMNLP 2024*, 2024.

[10] D. Lopez-Paz, L. Bottou, B. Schölkopf, and V. Vapnik. Unifying distillation and privileged information. In *ICLR Workshop*, 2016.

[11] X. Ning, Z. Lin, H. Yang, and Y. Wang. Skeleton-of-thought: Prompting llms for efficient parallel generation. *arXiv preprint arXiv:2307.15337*, 2023.

[12] A. Rodionov et al. Hogwild! inference: Parallel llm generation via concurrent attention. *arXiv preprint arXiv:2504.06261*, 2025.

[13] Alireza Salemi, Mihir Parmar, Palash Goyal, Yiwen Song, Jinsung Yoon, Hamed Zamani, Tomas Pfister, and Hamid Palangi. LLM-based multi-agent blackboard system for information discovery in data science. *arXiv preprint arXiv:2510.01285*, 2025.

[14] M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *arXiv preprint arXiv:1811.03115*, 2018.

[15] Z. Sun et al. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[16] V. Vapnik. Learning using privileged information: Similarity control and knowledge transfer. *Journal of Machine Learning Research*, 16, 2015.

[17] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. In *International Conference on Learning Representations*, 2025.

[18] G. Xiao et al. Sprint: Enabling interleaved planning and parallelized execution in large reasoning models. *arXiv preprint arXiv:2506.05745*, 2025.

[19] M. Yan et al. Decoding speculative decoding. *arXiv preprint arXiv:2402.01528*, 2024.

[20] L. Zhang, L. Fang, C. Duan, M. He, L. Pan, P. Xiao, S. Huang, Y. Zhai, X. Hu, P. S. Yu, and A. Liu. A survey on parallel text generation: From parallel decoding to diffusion language models. *arXiv preprint arXiv:2508.08712*, 2025.

[21] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62743–62767, 2024.

[22] Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.

A Training Objectives

Let Π^S denote the student planner logits and y_{plan} the latent planner-slot targets. The planner objective is cross-entropy over latent slots:

$$\mathscr{L}_{\text{plan}} = \text{CE}\big(\Pi^S, y_{\text{plan}}\big)\,, \tag{12}$$

with padded slots ignored by the label mask.

Let $\widehat{\mathbf{N}}^{\text{notes}}$ be the notes-head prediction, $\widehat{\mathbf{N}}^{\text{spec}}$ the speculative note prediction, and $\mathbf{N}^T$ the teacher notes tensor. The note-alignment losses are

$$\mathscr{L}_{\text{notes}} = \text{MSE}\Big(\widehat{\mathbf{N}}^{\text{notes}}, \mathbf{N}^T\Big)\,, \qquad \mathscr{L}_{\text{spec}} = \text{MSE}\Big(\widehat{\mathbf{N}}^{\text{spec}}, \mathbf{N}^T\Big)\,. \tag{13}$$

Once token-level supervision is active, the language-model losses are

$$\mathscr{L}_{\text{LM-CE}} = \text{CE}(\texttt{lm_logits}, \texttt{labels})_{m_{\text{labels}}}\,, \qquad \mathscr{L}_{\text{KD-LM}} = \text{KL}\big(p^S_{\text{LM}} \,\|\, p^T_{\text{LM}}\big)_{m_{\text{LM}}}\,. \tag{14}$$

Coverage supervision remains binary cross-entropy with logits over canonical plan items:

$$\mathscr{L}_{\text{cov}} = \text{BCEWithLogits}(\mathbf{c}, y_{\text{cov}})_{m_{\text{cov}}}\,, \tag{15}$$

where $\mathbf{c}$ are coverage logits, y_{cov} are binary ownership-aware coverage targets, and m_{cov} is the coverage mask.

Agreement supervision is reframed as **commit-readiness supervision**:

$$\mathscr{L}_{\text{ready}} = \text{BCE}\big(\mathbf{r}, y_{\text{ready}}\big)_{m_{\text{ready}}}\,, \tag{16}$$

where $\mathbf{r}$ are predicted readiness scores and y_{ready} indicates whether a provisional block had sufficient sibling information for safe commitment and continuation. When rollback annotations are available, premature or incoherent blocks are labeled as not ready rather than merely as post-hoc errors.

The total training loss is

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{plan}} + \mathscr{L}_{\text{notes}} + 0.5\,\mathscr{L}_{\text{spec}} + \mathscr{L}_{\text{LM-CE}} + \lambda_{\text{KD}}\,\mathscr{L}_{\text{KD-LM}} + \lambda_{\text{cov}}\,\mathscr{L}_{\text{cov}} + \lambda_{\text{ready}}\,\mathscr{L}_{\text{ready}}. \tag{17}$$

This objective matches the revised mechanism: the planner seeds the initial contract, note losses supervise the shared workspace, coverage supervises ownership consistency, and readiness supervises whether provisional parallel generation may safely continue.

Figure 1: Parallel Decoder Transformer Architecture

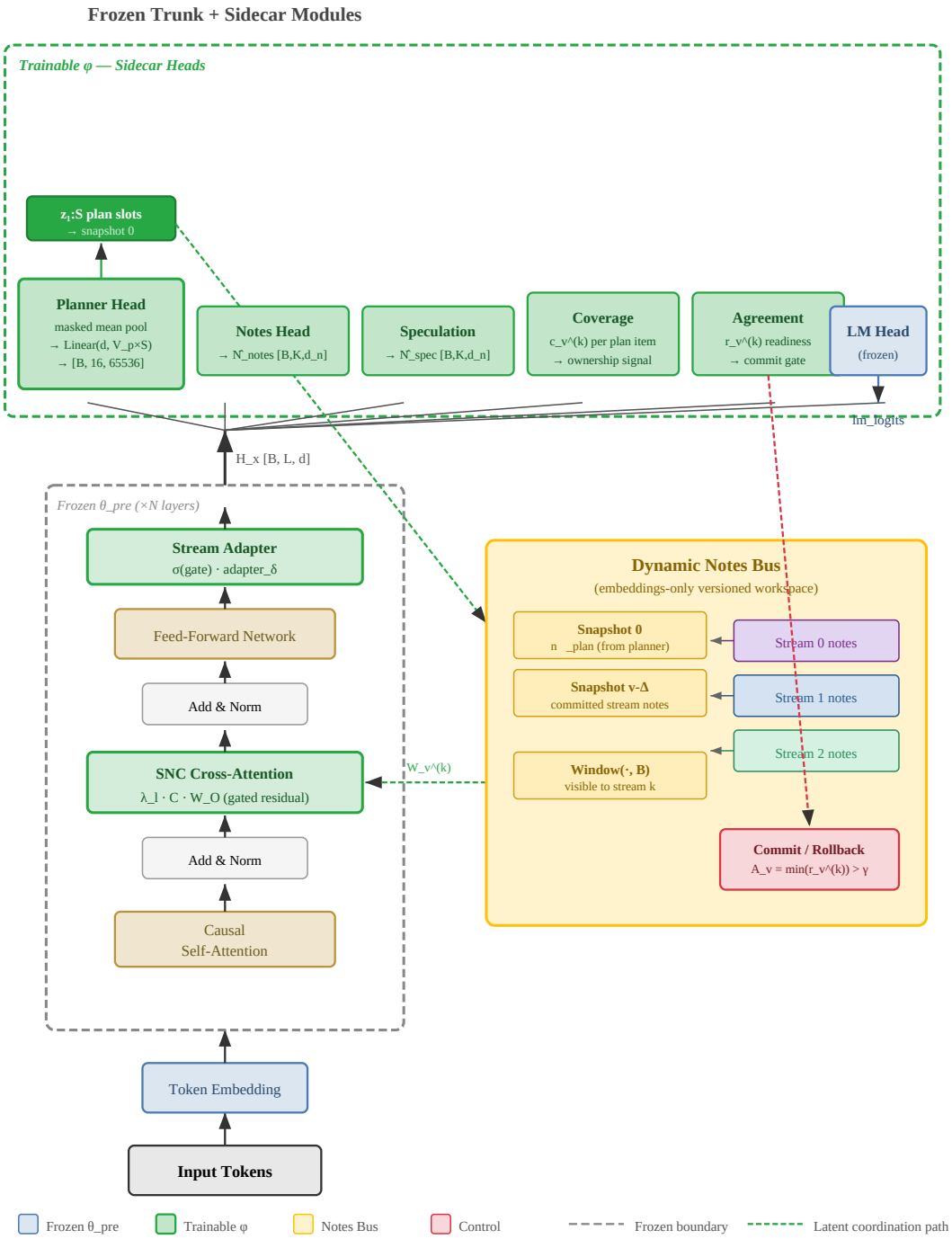

Figure 1: High-level architecture of the Parallel Decoder Transformer. The frozen trunk (left) is augmented with trainable sidecar modules (green): stream adapters and SNC cross-attention injected into selected transformer blocks, plus planner, notes, coverage, agreement, and speculation heads. The Dynamic Notes Bus (right) serves as the shared latent coordination workspace. Snapshot 0 is seeded by the planner before any stream emits tokens.

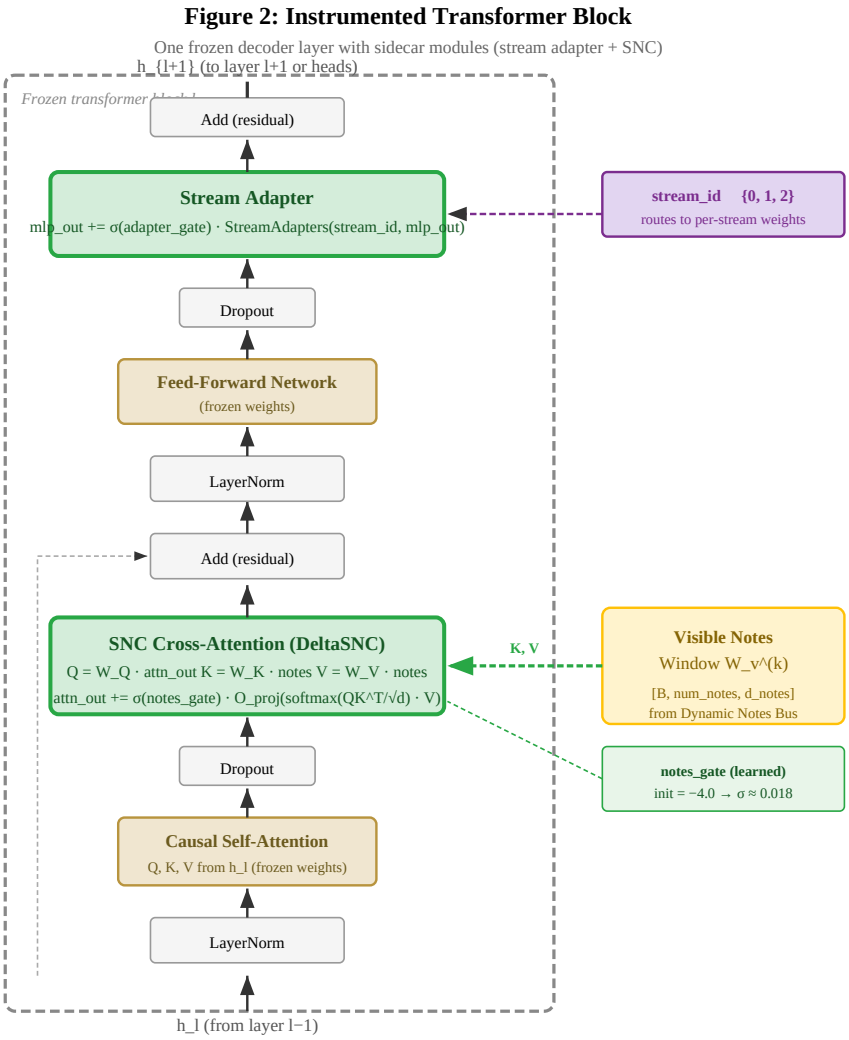

Figure 2: A single instrumented transformer block. After causal self-attention, SNC cross-attention injects a gated residual from the visible notes window. After the feed-forward network, the stream adapter adds a gated stream-specific residual. Both injection points use learned gates initialized near zero to preserve frozen trunk behavior early in training. All frozen weights (tan) are unchanged; only the sidecar modules (green) receive gradients.

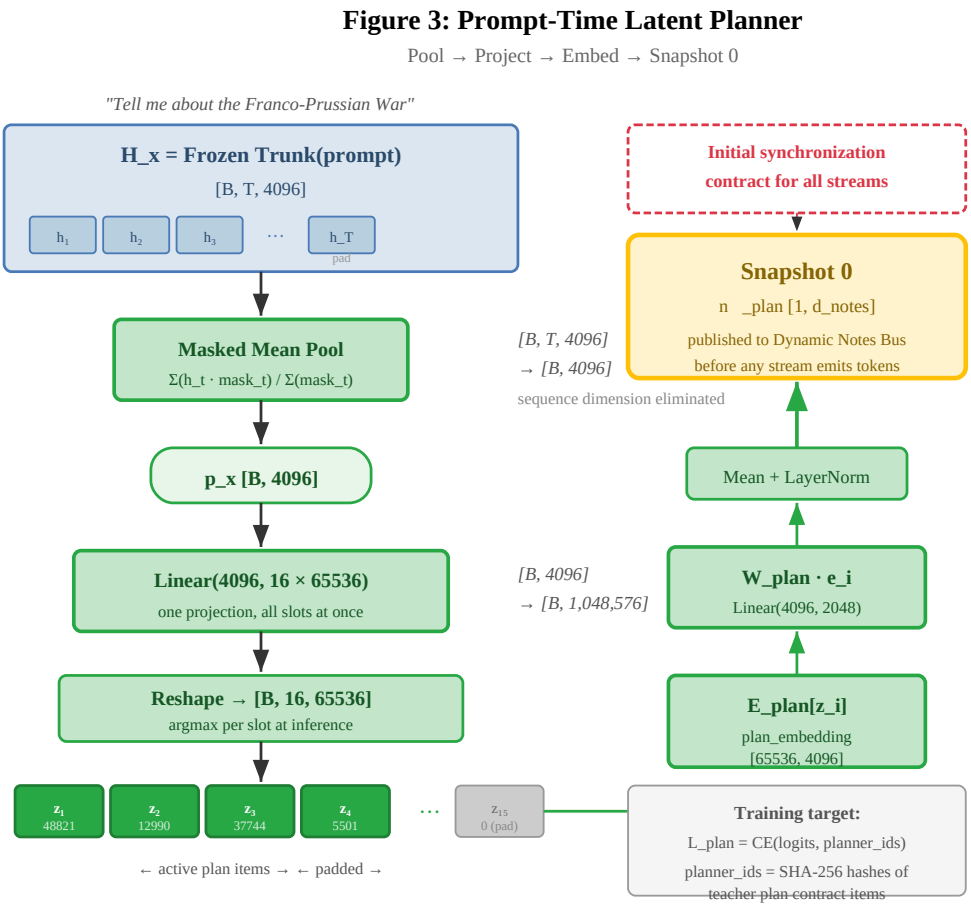

Figure 3: The prompt-time latent planner. Hidden states from the frozen trunk are masked-mean-pooled across the full sequence into a single vector, then projected in one linear pass into 16 slot logits over a 65,536-entry latent vocabulary. Each slot is argmax'd to select a latent plan item, re-embedded through the shared plan embedding matrix, projected into notes space, and published as snapshot 0. The planner operates on the complete prompt representation—not token by token—so it sees the full query before decomposing it into parallel plan items.

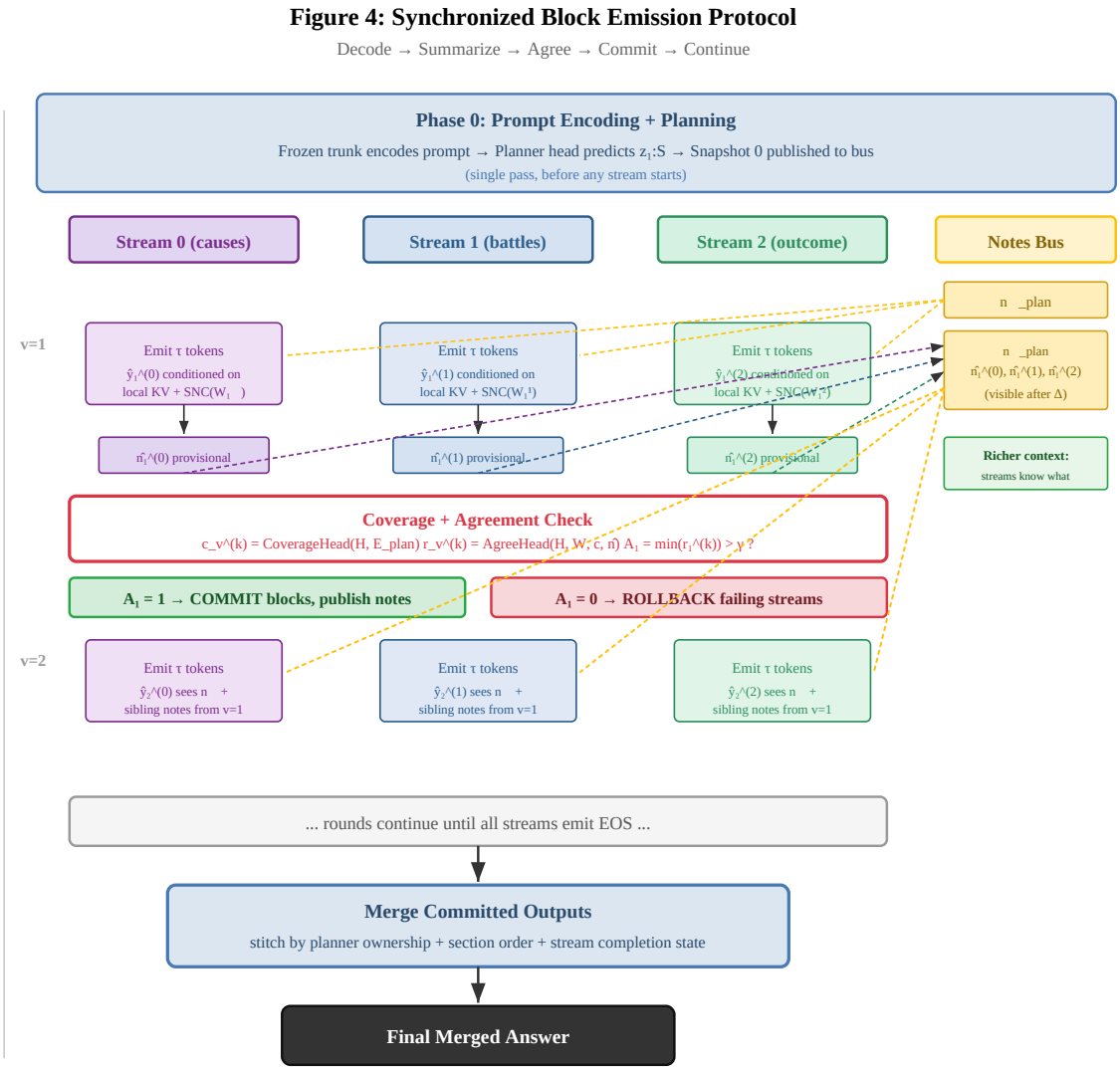

Figure 4: The synchronized block emission protocol across multiple rounds. After prompt encoding and planning (Phase 0), K streams decode in parallel, each emitting τ provisional tokens per round conditioned on the visible notes window via SNC. At each block boundary, streams emit provisional latent notes, coverage and agreement heads evaluate commit readiness, and the system either commits (advancing the frontier) or rolls back failing streams. Each successive round sees a richer notes window as committed sibling summaries become visible after delay Δ.